Book API Technical Report

1. Project overview

This project is a database-driven web API for managing and exploring book metadata. It was developed for the XJCO3011 Web Services and Web Data coursework using Django and Django REST Framework. The final system supports full CRUD operations, public dataset import, query filtering, rating-ranked search, weighted book recommendations, collection statistics, authentication for write operations, and interactive API documentation.The idea of this project is a book metadata and recommendation API based on a public dataset.

2. Technology stack and rationale

Programming language

Python was chosen because it allows fast development, has strong ecosystem support for web APIs and data handling, and fits naturally with the technologies introduced in the module.

Frameworks and libraries

- Django provides the project structure, ORM, admin panel, and migration system.
- Django REST Framework provides serializers, viewsets, request parsing, validation, and the browsable API.
- drf-spectacular provides OpenAPI schema generation together with Swagger UI documentation.

This stack was selected because it offers a good balance between speed of development, maintainability, and clarity for coursework presentation.

Database

SQLite was used as the SQL database. It is lightweight, easy to run locally, and fully suitable for the coursework requirement of a relational database-backed API.

Figure 1. Illustration of database

3. Public dataset integration

The project integrates a public CSV dataset stored at archive/books.csv. The dataset used for this coursework is from *Goodreads-books* (Soumik, 2020).

According to the dataset description, it provides Goodreads-style book metadata. The current imported CSV contains book metadata fields including: title, authors, average rating, ISBN and ISBN13, language code, number of pages, ratings count, text reviews count, publication date, publisher, book ID.

The Django Book model stores the fields that are most useful for the API demonstration. Since the new dataset does not include genre, description, or thumbnail values, imported records use Uncategorised as the genre and leave description and thumbnail blank unless edited manually through the API or admin interface.

To support this dataset, a custom Django management command was implemented:

```
python manage.py import_books_dataset --replace
```

This command imports the dataset into the SQLite database and can optionally replace existing records. Using a public dataset makes the project better aligned with the coursework brief and makes the API more realistic to demonstrate.

4. Data model

The application uses one main model: Book. Each book stores: bookid, title, author, genre, published_year, description, pages, publisher, language, average_rating, ratings_count, thumbnail

The id field remains the normal Django database primary key, while bookid is a separate sequential coursework-facing identifier. After dataset import, bookid is renumbered from 1 to the number of stored records so that books can be looked up using a cleaner demonstration ID without changing the database primary key. The model is still simple enough to explain clearly, but it is now rich enough to support metadata browsing, filtering, rating-ranked search, and weighted similarity-based recommendation behaviour based on the imported public dataset.

5. API design

The final API endpoints are:

- GET /api/books/
- POST /api/books/
- GET /api/books/{id}/
- GET /api/books/by-bookid/{bookid}/
- PUT /api/books/{id}/
- DELETE /api/books/{id}/
- GET /api/books/stats/
- GET /api/books/search/
- GET /api/books/recommendations/similar/

This API design keeps the core CRUD structure compact while adding functionality that better fits the book dataset use case. Genre-based lookup is still available through filtering on the main collection endpoint for manually categorised records, while rating-ranked search is handled through a dedicated search endpoint and similarity-based recommendations are handled through a separate weighted recommendation endpoint.

6. Search logic

The search endpoint is designed for user-driven retrieval from the current Goodreads CSV dataset. It allows results to be narrowed by title, author, publisher, language, and minimum rating, then ranks matching books by `average_rating` first and uses `ratings_count` as a secondary signal when ratings are tied. The endpoint returns up to the requested limit, so higher-rated and better-supported matches appear before weaker ones.

7. Recommendation logic

The project also includes a separate similarity-based recommendation endpoint so that recommendation is not confused with ordinary search. The endpoint accepts a seed book identified by bookid, then scores other books against that seed.

The weighted scoring formula is:

```
base_score =
    0.40 * title_score + 0.15 * author_score + 0.10 * publisher_score + 0.15 *
    language_score + 0.15 * rating_score + 0.05 * year_score
```

Where:

- title_score combines title keyword overlap and title sequence similarity,
- author_score rewards shared author information but does not dominate the final result,
- publisher_score rewards matching publishers,
- language_score rewards books in the same language,
- rating_score combines average rating with rating-count confidence,
- year_score rewards books published in a similar period.

To improve result diversity, the final list is not produced by raw score alone. Instead, the system applies:

- ***author repetition penalties,***
- ***title duplication penalties for very similar titles.***

This means the system avoids filling the top results with too many books by the same author or too many near-duplicate title variants. This produces a recommendation list that is more balanced and more useful than a simple same-author lookup.

8. Authentication and permissions

The API uses Basic Authentication for write operations. Public GET requests can be used without logging in, while POST, PUT, and DELETE require authentication.
This design was chosen because:

- it is easy to demonstrate in Swagger,
- it keeps read-only exploration simple,
- it protects database-modifying operations,
- it matches the coursework requirement to document authentication where applicable.

9. Validation and error handling

Validation is handled in the serializer layer to ensure that required fields are not blank and that years and ratings stay within valid ranges. The API returns standard REST status codes, including `200`, `201`, and `204` for successful operations, and `400`, `401`, and `404` for validation errors, unauthorised write attempts, and missing resources.

10. Documentation and version control

The project includes multiple forms of documentation: *README.md* for setup instructions and project overview, *api_docs.md* for manual API documentation, generated OpenAPI schema in *schema.yml*, *Swagger UI* for live interactive documentation, a PDF export of the API documentation for submission.

Each module update is promptly uploaded to GitHub, and the content of each push update is clearly noted, ensuring that the project remains under good version control as shown in the figure 2：

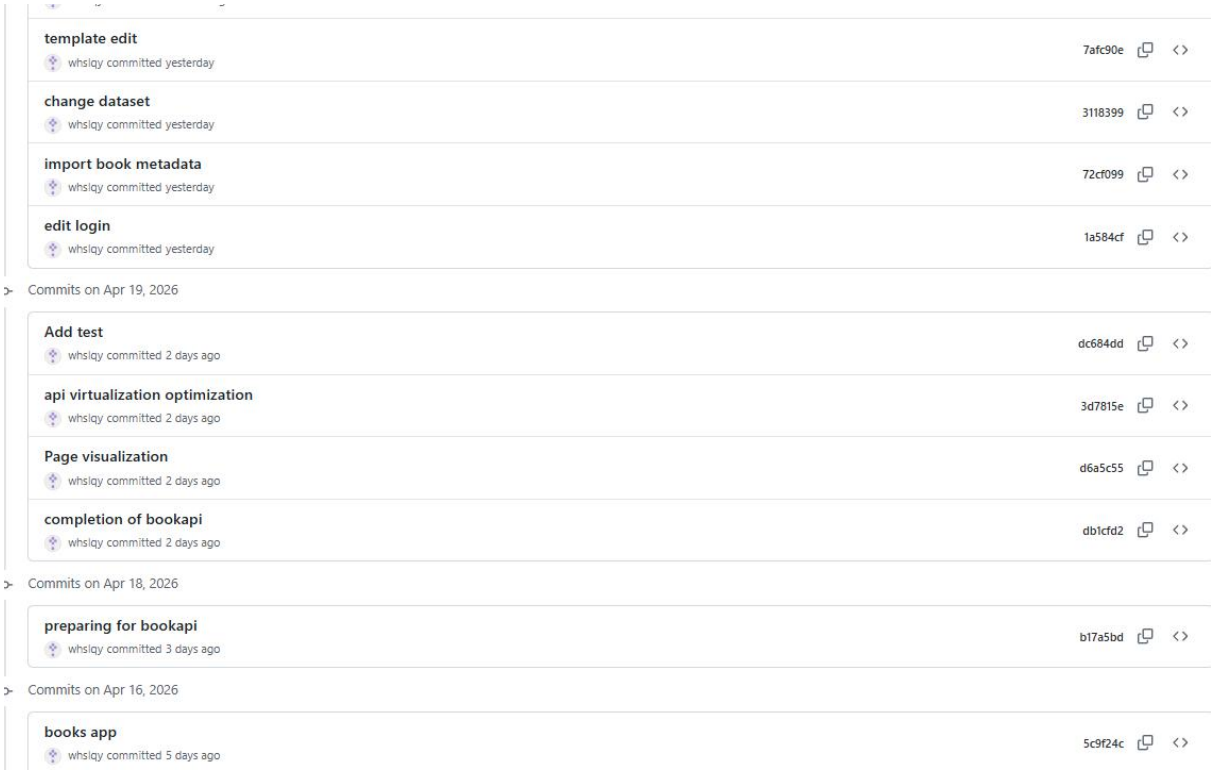

Figure 2. Illustration of version control

11. Testing approach

Automated API tests were implemented using Django REST Framework test tools to verify the main system behaviour, including CRUD operations, filtering, search, statistics, recommendations, validation, authentication, and Swagger UI availability. These tests were used throughout development to confirm that the API remained stable after design changes.

12. Challenges and Limitations

Several important design decisions shaped the final system. The project was extended from a simpler CRUD-only API into a public-dataset book metadata API so that it aligned more closely with the coursework brief. A separate genre analytics endpoint was explored and later removed because it overlapped with the overall stats endpoint and made the interface less clear. The search and recommendation features were also separated so that direct metadata retrieval and weighted similarity-based recommendation would serve distinct purposes within the API.

The project also has some limitations. It uses a single main model rather than a more complex relational structure, it is designed primarily for local execution and coursework demonstration rather than full production deployment, authentication is basic and not suitable for a large real-world platform, recommendation quality still depends on manually chosen weights rather than learned user behaviour, pagination is not currently implemented, and search and recommendation quality remain limited by the

completeness and quality of the imported dataset. These constraints are acceptable for the coursework scope, but they also identify clear extension points for future development.

13. Future improvements

Possible future improvements include:

- adding related models such as reviews, users, reading lists, or favourites,
- tuning weights using evaluation experiments or user feedback,
- expanding recommendation features with richer metadata or user interaction history,
- introducing pagination for larger result sets,
- deploying the project to a hosted platform,
- replacing Basic Authentication with token-based or JWT-based authentication,
- importing and merging additional public datasets for richer metadata.

14. References

Soumik. 2020. *Goodreads-books*. [Online]. [Accessed 20 April 2026]. Available from: https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks

Appendix

Deployment links

The deployed application is available at:

- https://whslqy.pythonanywhere.com/
- https://whslqy.pythonanywhere.com/api/docs/swagger/

Admin id:admin

Admin password: Admin12345!

Github links：

https://github.com/whslqy/web_cwk1 （containing code and API documentation）

Generative AI declaration

Generative AI was used as a development support tool throughout this project. Its contribution was not limited to code generation, but also included planning, iterative design refinement, data processing support, algorithm exploration, and documentation alignment.

AI was first used to help plan the overall API structure and refine endpoint design so that the final system matched the coursework brief more closely. This was particularly useful when the project evolved from a simpler CRUD-based API into a richer book metadata system with search, recommendation, statistics, and documentation features.

AI also made an important contribution to dataset handling. Because the project involved changing the dataset twice during development, AI was used to help analyse the new CSV structure, generate and revise the import script, and suggest preprocessing steps for inconsistent or irregular rows. This made it easier to adapt the database import pipeline to the updated Goodreads dataset and reduced the amount of manual trial-and-error during data cleaning. The dataset import process supported by AI is illustrated in Figure 3.

Figure 3. Importing the dataset with AI

In addition, AI contributed to the design of the recommendation system. It was used to explore alternative similarity signals and ranking strategies, including title text similarity, metadata-based weighting, and diversity penalties. These suggestions helped shape the final recommendation method, which balances title, author, publisher, language, rating quality, and publication year while reducing repetitive outputs from the same author or near-duplicate titles. The optimisation process for the recommendation system is illustrated in Figure 4.

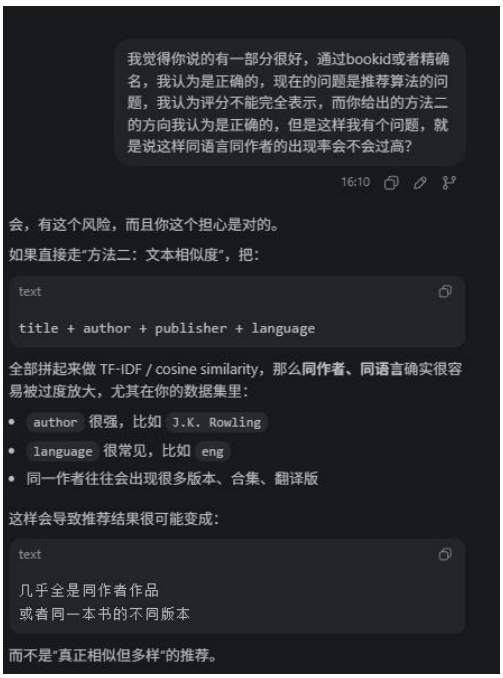

Figure 4(a) Optimizing the book recommendation system with AI

Figure 4(b) Optimizing the book recommendation system with AI

AI was also used to improve code quality and consistency. This included support with indentation, formatting, naming consistency, and general code organisation, which helped keep the implementation cleaner and easier to maintain. Finally, AI was used to help align the codebase with the supporting documentation, including the technical report, README, and API documentation, so that the submitted materials reflected the current implementation accurately.

Although AI provided substantial development support, the final design choices, feature selection, dataset decisions, weighting strategy, and documentation content were reviewed and adapted to fit the coursework requirements and the specific needs of the project.